

Лабораторная работа № 3

Анализ сетевого трафика с помощью tcpdump и Wireshark

Продолжительность

2 занятия по 90 минут.

Среда выполнения

- VirtualBox;
 - Ubuntu Server или Debian Server;
 - сервер web01;
 - клиент client01;
 - Wireshark на основном компьютере или отдельной VM с графическим интерфейсом.
-

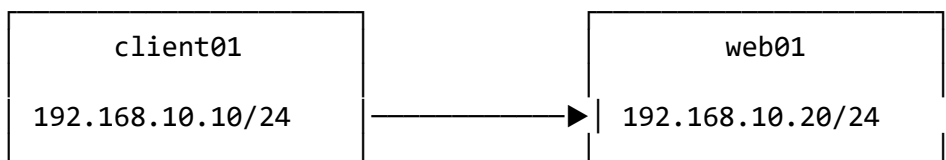
1. Цель работы

Научиться использовать пакетный захват как инструмент диагностики сетевых служб Linux-сервера.

В ходе работы необходимо:

- выбрать правильную точку захвата;
 - определить сетевой интерфейс;
 - применять capture-фильтры tcpdump;
 - записывать трафик в PCAP;
 - открывать PCAP в Wireshark;
 - анализировать ARP и DNS;
 - находить TCP handshake;
 - отличать открытый порт, закрытый порт и firewall drop;
 - анализировать HTTP-запрос и ответ;
 - оформлять технический вывод по захвату.
-

2. Исходная схема



tcpdump, curl, dig

Nginx, DNS, tcpdump

Обе виртуальные машины подключены к одной сети VirtualBox:

Internal Network: LAB-NET

Таблица адресации

Устройство	IP-адрес	Маска
client01	192.168.10.10	255.255.255.0
web01	192.168.10.20	255.255.255.0

Имя сервера:

web01.company.test

DNS-запись:

web01.company.test → 192.168.10.20

3. Исходное состояние

На web01 должны работать:

- Nginx на TCP 80;
- DNS-служба;
- SSH для административного доступа.

На client01 должны работать:

- разрешение имени web01.company.test;
- доступ к сайту по IP и имени.

Проверка с client01:

```
ping -c 4 192.168.10.20
getent hosts web01.company.test
curl -I http://192.168.10.20/
curl -I http://web01.company.test/
```

Ожидаемый HTTP-ответ:

HTTP/1.1 200 OK

4. Подготовка программ

На обеих VM установить tcpdump:

```
sudo apt update
sudo apt install -y tcpdump
```

На client01 установить диагностические программы:

```
sudo apt install -y curl dnsutils iproute2
```

На web01:

```
sudo apt install -y nginx curl dnsutils nftables python3
```

Проверить версию tcpdump:

```
tcpdump --version
```

Посмотреть доступные интерфейсы захвата:

```
sudo tcpdump -D
```

5. Базовые задания

Задание 1. Определить точку захвата и интерфейс

На client01 смотрим сетевые интерфейсы:

```
ip -br link
ip -br addr
```

Определяем локальное решение маршрутизации до сервера:

```
ip route get 192.168.10.20
```

Пример вывода:

```
192.168.10.20 dev enp0s3 src 192.168.10.10
```

Из вывода определяем:

Параметр	Значение
Destination	192.168.10.20
Interface	enp0s3
Source IP	192.168.10.10

В дальнейших командах вместо enp0s3 нужно использовать фактическое имя интерфейса своей VM.

Создаём переменную:

```
IFACE=enp0s3
```

Проверяем:

```
echo "$IFACE"
```

Вывод

Команда `ip route get` показывает, какой интерфейс и исходный IP выберет локальная система.

Она не доказывает, что пакет действительно достигнет сервера.

Задание 2. Захватить ARP Request и ARP Reply

На `client01` смотрим текущую таблицу соседей:

```
ip neigh show
```

Удаляем сохранённую ARP-запись сервера:

```
sudo ip neigh del 192.168.10.20 dev "$IFACE" 2>/dev/null
```

Если записи не было, команда может вывести ошибку. Это не мешает выполнению задания.

В первом терминале запускаем захват:

```
sudo timeout 15 tcpdump \  
-i "$IFACE" \  
-nn \  
-e \  
arp
```

Во втором терминале отправляем ping:

```
ping -c 1 192.168.10.20
```

Пример ARP-запроса:

```
ARP, Request who-has 192.168.10.20 tell 192.168.10.10
```

Пример ответа:

```
ARP, Reply 192.168.10.20 is-at 08:00:27:aa:bb:cc
```

Повторно смотрим таблицу соседей:

```
ip neigh show 192.168.10.20
```

Возможный результат:

```
192.168.10.20 dev enp0s3 lladdr 08:00:27:aa:bb:cc REACHABLE
```

Что нужно определить

- кто отправил ARP Request;
 - какой IP-адрес искал клиент;
 - какой MAC-адрес сообщил сервер;
 - почему ARP выполняется до ICMP Echo.
-

Задание 3. Захватить DNS-запрос и ответ

В первом терминале client01 запускаем:

```
sudo timeout 20 tcpdump \  
-i "$IFACE" \  
-nn \  
-vv \  
'host 192.168.10.20 and port 53'
```

Во втором терминале выполняем прямой запрос:

```
dig @192.168.10.20 web01.company.test
```

В захвате должны появиться:

- DNS Query от клиента к серверу;
- DNS Response от сервера к клиенту.

Пример запроса:

```
192.168.10.10.53042 > 192.168.10.20.53
```

Пример ответа:

```
192.168.10.20.53 > 192.168.10.10.53042
```

Обратить внимание:

- порт клиента является временным;
- Destination Port запроса — 53;
- Source Port ответа — 53.

Проверяем системное разрешение имени:

```
getent hosts web01.company.test
```

Вывод

Фильтр:

```
port 53
```

выбирает пакеты, в которых порт 53 является портом источника или назначения.

Задание 4. Захватить успешное TCP-соединение

На web01 проверяем Nginx:

```
systemctl is-active nginx  
sudo ss -lntp | grep ':80'
```

На client01 запускаем захват:

```
sudo timeout 20 tcpdump \  
-i "$IFACE" \  
-nn \  
-tttt \  
'host 192.168.10.20 and tcp port 80'
```

Во втором терминале выполняем:

```
curl -I http://192.168.10.20/
```

В захвате находим начало TCP-соединения:

```
Client → Server  SYN  
Server → Client  SYN, ACK  
Client → Server  ACK
```

Это TCP Three-Way Handshake.

После обмена данными обычно наблюдается завершение:

```
FIN, ACK  
ACK  
FIN, ACK  
ACK
```

Что доказывает handshake

Успешный handshake подтверждает:

- IP-связность;
- прохождение TCP;
- доступность TCP-порта;
- наличие процесса или механизма, принимающего соединение.

Handshake не доказывает, что приложение вернуло правильную веб-страницу.

Задание 5. Просмотреть HTTP-запрос и ответ

Запускаем кратковременный захват с ASCII-выводом:

```
sudo timeout 15 tcpdump \  
-i "$IFACE" \  
-nn \  
-s 0 \  
-A \  
'host 192.168.10.20 and tcp port 80'
```

Во втором терминале:

```
curl http://web01.company.test/
```

В выводе ищем HTTP-запрос:

```
GET / HTTP/1.1  
Host: web01.company.test
```

И HTTP-ответ:

```
HTTP/1.1 200 OK
```

Важно

Ключ -A может показать:

- URL;
- заголовки;
- cookies;
- данные формы;
- данные открытых протоколов.

В рабочей сети такой вывод выполняется только при необходимости и с разрешения.

Задание 6. Записать трафик в PCAP

Создаём защищённый каталог:

```
sudo install -d -m 700 /root/pcap
```

Запускаем запись:

```
sudo timeout 30 tcpdump \  
-i "$IFACE" \  
-nn \  
-s 0 \  
-c 100 \  
-w /root/pcap/http-success.pcap \  
'host 192.168.10.20 and tcp port 80'
```

Пока захват работает, во втором терминале выполняем:

```
curl http://web01.company.test/
```

Проверяем файл:

```
sudo ls -lh /root/pcap/http-success.pcap  
sudo file /root/pcap/http-success.pcap
```

Ограничиваем доступ:

```
sudo chmod 600 /root/pcap/http-success.pcap
```

Читаем файл через tcpdump:

```
sudo tcpdump -nn -r /root/pcap/http-success.pcap
```

Читаем только HTTP-порт:

```
sudo tcpdump \  
-nn \  
-r /root/pcap/http-success.pcap \  
'tcp port 80'
```

Что нужно зафиксировать

- интерфейс захвата;
 - capture point;
 - capture filter;
 - время захвата;
 - имя PCAP-файла;
 - права доступа;
 - количество записанных пакетов.
-

Задание 7. Открыть PCAP в Wireshark

Передать PCAP на компьютер с Wireshark:

- через защищённую общую папку VirtualBox;
- через SCP;
- через SFTP;
- другим способом, указанным преподавателем.

PCAP нельзя отправлять в общий чат или сохранять в общедоступном каталоге без проверки содержимого.

Открыть файл:

```
http-success.pcap
```

Применить display filters.

Весь TCP-порт 80

```
tcp.port == 80
```


Пакеты сервера

`ip.addr == 192.168.10.20`

Первый SYN без ACK

`tcp.flags.syn == 1 && tcp.flags.ack == 0`

HTTP

`http`

Только HTTP-запросы

`http.request`

Только HTTP-ответы

`http.response`

Найти пакеты:

1. SYN;
2. SYN/ACK;
3. ACK;
4. HTTP GET;
5. HTTP 200 OK;
6. FIN/ACK.

Записать номера этих пакетов в отчёт.

Главное различие

Capture filter:

`tcp port 80`

определяет, какие пакеты попадут в PCAP.

Display filter:

`tcp.port == 80`

определяет, какие из уже записанных пакетов показываются в Wireshark.

Задание 8. Исследовать закрытый TCP-порт

На `web01` проверяем, что TCP 8081 не слушается:

```
sudo ss -lntp | grep ':8081'
```

Нормальный результат — отсутствие вывода.

На `client01` запускаем:

```
sudo timeout 15 tcpdump \  
-i "$IFACE" \  
-nn \  
'host 192.168.10.20 and tcp port 8081'
```

Во втором терминале:

```
curl \  
--connect-timeout 5 \  
http://192.168.10.20:8081/
```

Ожидаемая ошибка:

Connection refused

В захвате обычно наблюдается:

```
Client → Server  SYN  
Server → Client  RST, ACK
```

Вывод

RST является реально полученным TCP-пакетом.

Он показывает активный отказ:

- порт закрыт;
- приложение не слушает;
- firewall работает в режиме reject.

Для определения точной причины дополнительно проверяются:

```
sudo ss -lntp  
sudo nft list ruleset  
sudo ufw status verbose
```

Задание 9. Исследовать Firewall Drop

Работа выполняется через консоль VirtualBox, а не через соединение, которое может быть потеряно.

На web01 запускаем тестовый HTTP-сервер:

```
nohup python3 -m http.server 8080 \  
--bind 0.0.0.0 \  
>/tmp/http8080.log 2>&1 &
```

Сохраняем PID:

```
echo $! | sudo tee /run/http8080.pid
```

Проверяем порт:

```
sudo ss -lntp | grep ':8080'
```

С client01 проверяем доступ:

```
curl -I http://192.168.10.20:8080/
```

Ожидается HTTP-ответ.

Создаём отдельную временную таблицу nftables:

```
sudo nft add table inet lab_capture
```

Создаём цепочку:

```
sudo nft \
  'add chain inet lab_capture input { type filter hook input priority 0; policy
  accept; }'
```

Добавляем точечный Drop только для TCP 8080:

```
sudo nft add rule inet lab_capture input tcp dport 8080 drop
```

Проверяем правило:

```
sudo nft list table inet lab_capture
```

На client01 запускаем захват:

```
sudo timeout 20 tcpdump \
  -i "$IFACE" \
  -nn \
  'host 192.168.10.20 and tcp port 8080'
```

Во втором терминале:

```
curl \
  --connect-timeout 10 \
  http://192.168.10.20:8080/
```

Ожидается timeout.

В захвате обычно наблюдаются:

```
SYN
SYN Retransmission
SYN Retransmission
```

Ответа от сервера нет.

Удаляем временную таблицу firewall:

```
sudo nft delete table inet lab_capture
```

Повторно проверяем доступ:

```
curl -I http://192.168.10.20:8080/
```

Останавливаем тестовый сервер:

```
sudo kill "$(cat /run/http8080.pid)"  
sudo rm -f /run/http8080.pid
```

Сравнение

Сценарий	Пакетный признак
Открытый порт	SYN → SYN/ACK → ACK
Закрытый порт	SYN → RST/ACK
Firewall Drop	Повторные SYN без ответа

Задание 10. Выполнить итоговый анализ неизвестной неисправности

Преподаватель создаёт один из сценариев:

1. DNS возвращает неправильный адрес.
2. DNS-запрос уходит без ответа.
3. Nginx остановлен.
4. TCP-порт закрыт.
5. Firewall отбрасывает TCP 80.
6. Сервер получает запрос, но не отправляет ответ.
7. Клиент обращается к неправильному IP-адресу.

Студент получает только симптом:

client01 не открывает `http://web01.company.test`.

Этап 1. Проверить обычными командами

На client01:

```
getent hosts web01.company.test  
ip route get 192.168.10.20  
curl -v http://web01.company.test/
```

На web01:

```
systemctl status nginx --no-pager  
sudo ss -lntp | grep ':80'
```

Этап 2. Сформулировать гипотезу

Пример:

Клиент отправляет TCP SYN, но сервер не отвечает.

Этап 3. Выбрать Capture Point

Возможные варианты:

- client01;
- web01;
- обе ВМ.

Этап 4. Записать PCAP

Пример на клиенте:

```
sudo timeout 30 tcpdump \  
-i "$IFACE" \  
-nn \  
-s 0 \  
-w /root/pcap/final-case.pcap \  
'host 192.168.10.20 and (port 53 or tcp port 80)'
```

Во время захвата повторить:

```
curl -v http://web01.company.test/
```

Этап 5. Открыть PCAP в Wireshark

Применить фильтры:

```
dns
```

```
tcp.port == 80
```

```
dns.flags.rcode != 0
```

```
tcp.flags.reset == 1
```

```
tcp.analysis.retransmission
```

Этап 6. Сформулировать технический вывод

Вывод должен ссылаться на конкретные пакеты.

Пример:

Пакеты 12, 15 и 18 являются повторными SYN от client01 к TCP 80 web01. SYN/ACK или RST отсутствуют. Клиент отправляет запрос, но ответа не получает. Необходимо проверить серверный firewall и выполнить второй захват на web01.

Этап 7. Исправить подтверждённую причину

Изменяется только параметр, причина которого доказана.

Этап 8. Повторить захват

После исправления в PCAP должны появиться:

- успешный DNS-ответ;

- TCP handshake;
 - HTTP GET;
 - HTTP 200 OK.
-

6. Дополнительные задания

Дополнительное задание 1. Захватить DNS NXDOMAIN

На client01 запускаем:

```
sudo timeout 15 tcpdump \  
-i "$IFACE" \  
-nn \  
'host 192.168.10.20 and port 53'
```

Во втором терминале запрашиваем несуществующее имя:

```
dig @192.168.10.20 missing.company.test
```

Ожидаемый DNS-статус:

NXDOMAIN

В Wireshark открыть захват и применить:

```
dns.flags.rcode != 0
```

Вывод

NXDOMAIN и отсутствие ответа — разные ситуации:

- NXDOMAIN — сервер ответил, но имя не существует;
 - отсутствие ответа — запрос не получил DNS Response.
-

Дополнительное задание 2. Исследовать HTTPS без расшифровки

На web01 создаём временный сертификат:

```
sudo openssl req \  
-x509 \  
-newkey rsa:2048 \  
-nodes \  
-keyout /tmp/lab-key.pem \  
-out /tmp/lab-cert.pem \  
-days 1 \  
-subj '/CN=web01.company.test'
```

Запускаем тестовый TLS-сервер:

```
sudo openssl s_server \  
-accept 8443 \  
-cert /tmp/lab-cert.pem \  
-key /tmp/lab-key.pem \  
-www
```

На client01 в отдельном терминале запускаем захват:

```
sudo timeout 20 tcpdump \  
-i "$IFACE" \  
-nn \  
-s 0 \  
-w /root/pcap/https-test.pcap \  
'host 192.168.10.20 and tcp port 8443'
```

В другом терминале:

```
curl -k https://192.168.10.20:8443/
```

Открыть PCAP в Wireshark.

Применить:

```
tcp.port == 8443
```

```
tls
```

В захвате обычно видны:

- IP-адреса;
- TCP-порт;
- TCP handshake;
- TLS handshake;
- размеры и время пакетов.

HTTP-содержимое в открытом виде не отображается.

После проверки остановить openssl s_server:

```
Ctrl+C
```

Удалить временные файлы:

```
sudo rm -f /tmp/lab-key.pem /tmp/lab-cert.pem
```

Дополнительное задание 3. Создать потери и найти Retransmission

Работу выполнять через консоль VirtualBox.

На client01 добавить искусственную потерю 20% пакетов:

```
sudo tc qdisc add dev "$IFACE" root netem loss 20%
```

Проверить настройку:

```
tc qdisc show dev "$IFACE"
```

Запустить захват:

```
sudo timeout 30 tcpdump \  
  -i "$IFACE" \  
  -nn \  
  -s 0 \  
  -w /root/pcap/loss-test.pcap \  
  'host 192.168.10.20 and tcp port 80'
```

Во втором терминале несколько раз выполнить:

```
curl http://192.168.10.20/ >/dev/null
```

Обязательно удалить искусственную потерю:

```
sudo tc qdisc del dev "$IFACE" root
```

Открыть PCAP в Wireshark.

Применить:

```
tcp.analysis.retransmission
```

```
tcp.analysis.duplicate_ack
```

Важно

Retransmission и Duplicate ACK — аналитические признаки Wireshark, а не отдельные TCP-флаги.

Единичное событие не доказывает причину без проверки:

- Sequence Number;
 - Acknowledgment Number;
 - времени;
 - полноты захвата;
 - точки захвата.
-

7. Основные команды

Интерфейсы и маршрутизация

```
ip -br link  
ip -br addr  
ip route  
ip route get <destination>  
ip neigh show
```


Захват

```
sudo tcpdump -D
sudo tcpdump -i <interface>
sudo tcpdump -i <interface> -nn
sudo tcpdump -i <interface> -nn -e arp
sudo tcpdump -i <interface> -nn 'port 53'
sudo tcpdump -i <interface> -nn 'tcp port 80'
```

Запись PCAP

```
sudo tcpdump \
-i <interface> \
-n \
-s 0 \
-w capture.pcap \
'<capture-filter>'
```

Чтение PCAP

```
sudo tcpdump -nn -r capture.pcap
```

Служба и порт

```
systemctl status nginx
sudo ss -lntp
sudo nft list ruleset
```

8. Требования к отчёту

В отчёте представить:

1. Название и цель работы.
2. Схему стенда и таблицу адресации.
3. Фактическое имя интерфейса захвата.
4. Результат `ip route get`.
5. ARP Request и ARP Reply.
6. DNS Query и DNS Response.
7. Номера пакетов TCP handshake.
8. Номера пакетов HTTP GET и HTTP 200.
9. Сравнение закрытого порта и firewall drop.
10. Один итоговый вывод по неизвестной неисправности.
11. Имя и права созданного PCAP.
12. Краткий вывод по работе.

Шаблон анализа PCAP

Поле	Значение
------	----------

Поле	Значение
Симптом	
Гипотеза	
Capture Point	
Интерфейс	
Capture Filter	
Время захвата	
Имя PCAP	
Номера пакетов	
Наблюдение	
Технический вывод	
Дополнительная проверка	
Результат после исправления	

В отчёт включаются только минимальные фрагменты, необходимые для доказательства вывода.

9. Чек-лист выполнения

- ☐ Определён интерфейс с помощью `ip route get`.
- ☐ Захвачены ARP Request и ARP Reply.
- ☐ Захвачены DNS Query и DNS Response.
- ☐ Найден TCP Three-Way Handshake.
- ☐ Найдены HTTP GET и HTTP 200.
- ☐ Создан PCAP-файл с ограниченным capture filter.
- ☐ PCAP хранится в каталоге с ограниченным доступом.
- ☐ PCAP открыт в Wireshark.
- ☐ Capture filter не перепутан с display filter.
- ☐ Закрытый порт подтверждён через RST/ACK.
- ☐ Firewall Drop подтверждён повторными SYN без ответа.
- ☐ После теста временное правило nftables удалено.
- ☐ После теста временный HTTP-сервер остановлен.
- ☐ Итоговая неисправность подтверждена пакетами.
- ☐ После исправления получены handshake и HTTP 200.

10. Контрольные вопросы

1. Почему точка захвата влияет на диагностический вывод?
2. Что показывает `ip route get` и чего эта команда не доказывает?
3. Чем `capture filter` отличается от `display filter`?
4. Почему слишком узкий `capture filter` может помешать диагностике?
5. Как по захвату отличить открытый порт, закрытый порт и `firewall drop`?
6. Чем `RST` отличается от `timeout`?
7. Какие признаки видны в открытом HTTP?
8. Какие данные обычно остаются видимыми при анализе HTTPS?